

TECHNICAL REPORT · 2026



AgriAI

AI-Powered Smart Farming Assistant
System Architecture, Functionality & Use Cases

VERSION

1.0.0

AUTHOR

Vinit080

REPOSITORY

github.com/Vinit080/AgriAI

DATE

June 2026

3

AI MODELS

96.7%

DISEASE ACCURACY

54K

TRAINING IMAGES

38

DISEASE CLASSES

Executive Summary

AgriAI is a full-stack, production-grade web application that brings deep learning and machine learning to the fingertips of farmers, agronomists, and agricultural researchers. The platform integrates three independent AI engines into a single, secure, multi-lingual interface.



Fertilizer Prediction

Random-Forest regression model predicts optimal N, P, K requirements with linear-programming cost optimization.



Seed Quality Analysis

EfficientNetB0 CNN classifies seed images into High, Average, or Poor quality tiers with Grad-CAM heatmaps.



Crop Disease Detection

MobileNetV2 fine-tuned on 54,000 PlantVillage images across 38 disease classes at 96.7% accuracy.

Technology Stack

AgriAI is built on a modern Python backend with a secure Flask web framework, backed by TensorFlow/Keras for deep learning, scikit-learn for classical ML, and SQLite for persistence.

Backend Framework

Flask 3.1.3

Flask-Login 0.6.3

Flask-SQLAlchemy 3.1.1

Flask-WTF 1.3.0

Flask-Limiter 4.1.1

Flask-Talisman

Werkzeug 3.1.8

Jinja2 3.1.6

python-dotenv 1.2.2

Flask serves as the WSGI application server. Flask-Talisman enforces HTTP security headers (CSP, HSTS, X-Frame-Options). Flask-Limiter provides rate limiting at both global and per-route levels. Flask-WTF adds CSRF protection across all POST endpoints.

Machine Learning & AI

TensorFlow 2.21.0

Keras (bundled)

scikit-learn 1.9.0

SHAP 0.52.0

MobileNetV2

EfficientNetB0

Grad-CAM (custom)

joblib 1.5.3

TensorFlow/Keras powers two deep learning models (MobileNetV2 for disease detection, EfficientNetB0 for seed quality). scikit-learn provides the Random Forest pipeline for fertilizer NPK prediction. SHAP (SHapley Additive exPlanations) generates feature-importance charts. Grad-CAM is implemented via a custom TensorFlow gradient-tape loop.

Data Science & Image Processing

NumPy 2.4.6

Pandas 3.0.3

SciPy 1.17.1

Pillow 11.3.0

OpenCV 4.13

Matplotlib 3.10.9

SciPy's linear programming solver (HiGHS method) minimizes fertilizer procurement cost given N, P, K targets. OpenCV performs Grad-CAM superimposition — the heatmap is composited over the original image using colourmap JET and saved to the static directory. PIL/Pillow handles image decoding, format validation, and resizing.

Database & Authentication

SQLite (default)

SQLAlchemy 2.0.50

Flask-Login

PBKDF2-SHA256 hashing

WTForms 3.2.2

Passwords are hashed using Werkzeug's PBKDF2:SHA256 with a 16-byte salt (never stored in plaintext). User sessions are managed server-side by Flask-Login. The DATABASE_URL environment variable allows swapping SQLite for PostgreSQL in production without code changes.

Frontend

[HTML5 / Jinja2 Templates](#)[Vanilla CSS](#)[Chart.js](#)[Font Awesome 6.4](#)[Plus Jakarta Sans](#)[Google Translate API](#)

The UI uses a dark-mode design system with glassmorphism cards, animated progress bars, and Grad-CAM heatmap overlays. Chart.js powers the dashboard's analytics charts (bar + doughnut). The language switcher integrates the Google Translate cookie-based API, supporting 7 Indian regional languages. PDF export is implemented via the native browser print dialog.

03 — SYSTEM ARCHITECTURE

System Architecture

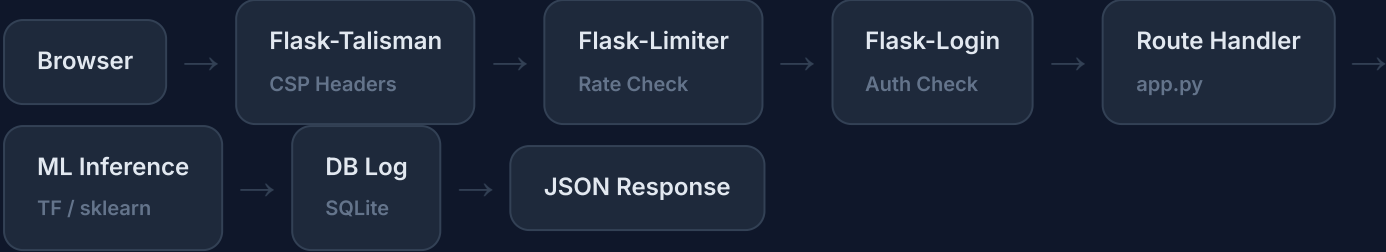
AgriAI follows a monolithic MVC architecture with clear separation between route handlers, ML inference, and data persistence layers.

Project Structure

```
AgriAI/
├─ app.py # Main Flask application, all routes & ML inference
├─ train_disease.py # PlantVillage MobileNetV2 trainer (2-phase)
├─ train_seed.py # Seed quality EfficientNetB0 trainer
├─ train_npk.py # NPK Random Forest trainer
├─ disease_model.h5 # Trained MobileNetV2 – 96.7% accuracy
├─ model.h5 # Trained EfficientNetB0 – 72.3% accuracy
├─ npk_model.joblib # Serialized sklearn Pipeline (RF + preprocessor)
├─ seed_class_names.json # Class index → label mapping
├─ Testing.csv # Fertilizer dataset (dashboard analytics)
├─ src/
│   └─ models.py # SQLAlchemy models: User, Farm, PredictionHistory
│       └─ ml/CNNModel.py # Original CNN training script (legacy)
├─ templates/ # Jinja2 HTML templates
│   └─ base.html # Master layout: sidebar, topbar, translate, scripts
│   └─ index.html # Dashboard with live analytics charts
│   └─ fertilizer.html # NPK form + results + SHAP chart
│   └─ seed.html # Image upload + Grad-CAM result
│   └─ disease.html # Leaf image upload + disease result
│   └─ login.html / register.html
│   └─ profile.html # Prediction history
│       └─ api_docs.html # Admin-only API documentation
└─ static/
```

```
|— css/style.css #Full design system (dark mode)
|— img/ # Static assets + generated Grad-CAM/SHAP images
```

Request Lifecycle



Database Schema

TABLE	COLUMN	TYPE	PURPOSE
user	id	INTEGER PK	Auto-increment primary key
	username	VARCHAR(50) UNIQUE	Login identifier
	password_hash	VARCHAR(256)	PBKDF2:SHA256 hash + salt
	is_admin	BOOLEAN	Role-based access control flag
farm	id	INTEGER PK	Farm record ID
	name	VARCHAR(100)	Farm name
	location	VARCHAR(100)	Geographic location
	area_hectares	FLOAT	Farm size in hectares
prediction_history	id	INTEGER PK	Record ID
	user_id	FK → user.id	Owner reference
	prediction_type	VARCHAR(20)	'Fertilizer' 'Seed' 'Disease'
	input_data	TEXT (500 chars)	Sanitized input summary
	result	VARCHAR(200)	Prediction output label

AI Models in Detail

Model 1 — Fertilizer NPK Predictor

RF

ALGORITHM

Multi

OUTPUT TARGET

3

OUTPUT VALUES

SHAP

EXPLAINABILITY

Architecture

A scikit-learn **Pipeline** consisting of a **ColumnTransformer** preprocessor (OneHotEncoder for categorical soil/season/crop columns + StandardScaler for numerical features) feeding into a **MultiOutputRegressor** wrapping a **RandomForestRegressor**. The pipeline is serialized with joblib.

Inputs

State, Year, Season, Crop, Area (ha), Production (MT), Annual Rainfall (mm), Average Temperature (°C), Soil pH, Soil Type

Outputs

Predicted N (kg/ha), P (kg/ha), K (kg/ha) requirements + linear-programming optimized fertilizer procurement plan (Urea, DAP, MOP quantities + INR cost)

Explainability (XAI)

A SHAP TreeExplainer generates a horizontal bar chart of the top-5 features driving the Nitrogen prediction, saved as a per-request unique PNG to prevent data leaks between concurrent users.

Model 2 — Crop Disease Detector

96.7%

VAL ACCURACY

38

CLASSES

54K

TRAIN IMAGES

224px

INPUT SIZE

Architecture

MobileNetV2 pretrained on ImageNet, used as a frozen feature extractor in Phase 1, then the top 30 layers are unfrozen for fine-tuning in Phase 2. Head: GlobalAveragePooling2D → Dense(38, softmax).

Training Dataset

PlantVillage dataset cloned locally (~54,000 colour images). 80/20 train/validation split with seed=123. Augmentation: RandomFlip, RandomRotation(0.1), RandomZoom(0.1), RandomContrast(0.1).

Training Phases

Phase 1 — Frozen backbone, Adam(lr=1e-3), 3 epochs → 94.8% val accuracy.

Phase 2 — Top-30 layers unfrozen, Adam(lr=1e-4), 3 epochs → 96.7% val accuracy.

Preprocessing

Images resized to 224×224 and normalized to [-1, 1] range (MobileNetV2 convention: pixel = pixel/127.5 - 1.0).

Model 3 — Seed Quality Classifier

72.3%

VAL ACCURACY

3

CLASSES

2000

IMAGES

128px

INPUT SIZE

Architecture

EfficientNetB0 pretrained on ImageNet. Head: GlobalAveragePooling2D → BatchNormalization → Dropout(0.3) → Dense(128, relu) → Dropout(0.2) → Dense(3, softmax).

Class Mapping

Raw dataset folders were merged into 3 quality tiers: **High** (pure seeds), **Average** (discolored seeds), **Poor** (broken + silkcut seeds combined). The class index mapping is persisted to `seed_class_names.json` and loaded at startup.

Grad-CAM Heatmap

A custom TensorFlow gradient-tape Grad-CAM implementation recursively searches for the last Conv2D layer across nested sub-models, handles MobileNetV2/EfficientNetB0 architectures, and generates a JET-colormapped superimposition saved per request with a UUID prefix.

05 — REST API REFERENCE

API Endpoints

All prediction endpoints require session authentication (cookie-based). Admin-only endpoints require `is_admin=True` in the database.

Authentication

POST**/login**

Authenticate with username + password. Sets session cookie. Rate-limited: 10 req/min.

POST**/register**

Create new user account. Min 8-char password, 50-char username limit. Rate-limited: 5 req/min.

GET**/logout**

Invalidate current session and redirect to home.

Core Prediction APIs

POST**/api/v1/predict/fertilizer**

JSON body. Returns N/P/K predictions, SHAP plot URL, and linear-programming cost schedule. Auth required. Rate: 20/hour.

POST**/api/v1/predict/seed**

Multipart/form-data with 'file' field (JPG/PNG/WEBP, max 5MB). Returns quality tier, confidence %, reasoning, practices, and Grad-CAM URL. Auth required. Rate: 20/hour.

POST**/api/v1/predict/disease**

Multipart/form-data with 'file' field. Returns disease class (1 of 38), confidence %, expert reasoning, and Grad-CAM URL. Auth required. Rate: 20/hour.

Utility APIs

GET**/api/stats**

Returns aggregated dashboard analytics (top crops by yield, fertilizer distribution). Cached at startup. Rate: 60/min.

GET**/api/v1/sensors/read**

IoT sensor mock endpoint. Returns randomized pH, temperature, rainfall, and soil type. Auth required. Rate: 30/min.

GET**/health**

Simple health check — returns {"status": "ok"}. No auth required.

FEATURE	DESCRIPTION	TECHNOLOGY
 Secure Authentication	Session-based login/register with PBKDF2:SHA256 password hashing, CSRF protection on all forms, PII-free failure logging	Flask-Login, Flask-WTF, Werkzeug
 NPK Fertilizer Analysis	10-field form with IoT sensor auto-fill. Predicts N, P, K requirements and computes minimum-cost procurement plan	RandomForest, SciPy linprog
 SHAP Explainability	Feature importance bar chart shows which farm inputs most influenced the Nitrogen prediction	SHAP TreeExplainer
 Seed Quality Analysis	Drag-and-drop image upload, quality tier classification (High/Average/Poor), confidence score, low-confidence warning	EfficientNetB0, Grad-CAM
 Disease Detection	38-class leaf disease classification from a single photo with expert remediation advice	MobileNetV2 (96.7% acc)
 Grad-CAM Heatmaps	Visual AI explanation showing which regions of seed/leaf images the model focused on, overlaid as a colour heatmap	TF GradientTape, OpenCV
 Analytics Dashboard	Bar and doughnut charts showing top-performing crops by average yield and fertilizer distribution from the dataset	Chart.js, Pandas
 Multi-Language Support	7-language switcher (English, Hindi, Marathi, Tamil, Telugu, Bengali, Gujarati) via Google Translate cookie API	Google Translate API
 PDF Export	One-click report download for fertilizer, seed, and disease results via the native browser print dialog	Browser Print API
 Prediction History	Per-user chronological log of all past predictions (type, input summary, result, timestamp) on the profile page	SQLAlchemy, SQLite
 Security Headers	Full Content-Security-Policy, X-Content-Type-Options, Referrer-Policy, and frame-ancestors enforcement	Flask-Talisman
 Rate Limiting	Global: 200/day, 50/hour. Per-route: Login 10/min, Predict 20/hour, Sensors 30/min	Flask-Limiter
 Magic-Byte Validation	Uploaded images verified against actual binary content (not just extension) to prevent malicious file uploads	Pillow PIL.verify()
 Admin Role	Database-driven admin flag unlocks API documentation page (not a hardcoded username comparison)	SQLAlchemy RBAC

Use Cases & User Scenarios

Primary Use Cases

1

Smallholder Farmer — Fertilizer Optimization

A farmer cultivating Wheat in Maharashtra enters crop details, soil pH, and recent rainfall. AgriAI predicts NPK requirements and recommends the cheapest combination of Urea, DAP, and MOP to meet those targets, minimizing input costs via linear programming. Output is downloaded as a PDF report.

2

Agronomist — Pre-Planting Seed Quality Check

An agricultural extension worker photographs a batch of corn seeds on a white background and uploads the image. AgriAI classifies them as "High Quality (Pure & Highly Suitable)" with 87% confidence and displays a Grad-CAM heatmap highlighting the morphological regions evaluated. The result is exported to PDF for field documentation.

3

Farm Manager — Early Disease Identification

A tomato farmer notices yellowing patterns on leaves and uploads a photo. The disease model identifies "Tomato Yellow Leaf Curl Virus" at 94% confidence, explains the visual diagnostic reasoning, and prescribes immediate isolation plus virucide application. The Grad-CAM overlay confirms which leaf patterns triggered the detection.

4

Regional Language User — Multi-Lingual Access

A farmer in Tamil Nadu selects "தமிழ் (Tamil)" from the language dropdown. The entire interface — including navigation, form labels, and AI results — is translated through Google Translate's cookie-based API, making the platform accessible without English literacy.

5

Researcher — Batch Analytics Review

An agricultural data scientist logs into the admin account, reviews the dashboard bar chart (top 5 crops by average yield from the Testing.csv dataset) and doughnut chart (fertilizer distribution), and uses the API Docs page to understand available endpoints for integration with external agronomic software.

6

IoT Integration — Sensor-Assisted Form Fill

A smart farm with connected soil sensors uses the `/api/v1/sensors/read` endpoint to auto-populate pH, temperature, and soil type fields in the fertilizer form. This eliminates manual data entry and reduces human error in field measurements.

7

Historical Audit — Prediction Traceability

A farm cooperative administrator reviews a member's profile page to audit all past predictions (type, date, crop, result) for compliance with government agricultural subsidy requirements. The history is stored per-user and includes timestamps.

8

Disease Screening at Scale — Extension Programs

An NGO running a crop health camp photographs leaf samples from 100+ farmers. Field workers use the disease detection module to screen all samples, flag diseased plants for expert review, and print PDF reports for each farmer — all without internet-dependent cloud APIs since inference runs locally.

Security Model

Authentication & Authorization

- ✓ Session-based login via Flask-Login
- ✓ PBKDF2:SHA256 password hashing (16-byte salt)
- ✓ CSRF tokens on all state-changing forms
- ✓ Role-based admin access via DB column
- ✓ Failed logins log anonymized SHA256 hash only (no PII)

Input Validation

- ✓ File type allowlist: PNG, JPG, JPEG, WEBP
- ✓ Magic-byte verification via PIL.verify()
- ✓ 5 MB upload size cap (Flask MAX_CONTENT_LENGTH)
- ✓ String length limits on all form fields
- ✓ Prediction history truncated to 500 chars

HTTP Security Headers

- ✓ Content-Security-Policy (full allowlist)
- ✓ X-Content-Type-Options: nosniff
- ✓ Referrer-Policy: strict-origin-when-cross-origin
- ✓ frame-ancestors: none (clickjacking prevention)
- ✓ CSRF token expiry: 1 hour

Rate Limiting & Resource Protection

- ✓ Global: 200 requests/day, 50/hour
- ✓ Auth endpoints: 5–10 requests/minute
- ✓ ML prediction routes: 20 requests/hour
- ✓ IoT sensor read: 30 requests/minute
- ✓ Unique filenames for all generated images (UUID)

Deployment Guide

Environment Variables

VARIABLE	REQUIRED	DEFAULT	PURPOSE
FLASK_SECRET_KEY	✔ Yes	—	Session signing key (min 32 bytes hex)
DATABASE_URL	No	sqlite:///farm.db	Database connection string
FLASK_ENV	No	production	Set to development to enable debug mode

Quick Start

```
# 1. Clone and install
git clone https://github.com/Vinit080/AgriAI
pip install -r requirements.txt

# 2. Configure environment
cp .env.example .env
# Edit .env: set FLASK_SECRET_KEY

# 3. (Optional) Train models
python train_disease.py # ~2.5hrs CPU → disease_model.h5
python train_seed.py # ~15mins CPU → model.h5
python train_npk.py # ~2mins → npk_model.joblib

# 4. Run
python app.py
```



AgriAI Technical Report

github.com/Vinit080/AgriAI · Generated June 2026